

frontend.md

Project: AI Business Automation Platform

Component: Frontend Server

Status: 最終需求、設計與實作計劃

Last Updated: 2026-07-07

Primary Goal: 建立一個專業、安全、具回應式設計的 SaaS frontend，讓使用者可以管理 AI agents、workflows、tools、approvals、knowledge、memory、evaluations、audits 及 organization settings。

1. 目的

frontend server 的存在，是為平台提供面向使用者的 web application。

它提供一個介面，讓使用者可以：

- 登入並管理工作階段。
- 查看主 dashboard。
- 建立、檢視及管理 AI agents。
- 設定 tools 與 integrations。
- 建立及執行 workflows。
- 審閱 workflow approvals。
- 管理 knowledge sources 與已索引文件。
- 檢視 memory records。
- 查看 evaluations 與品質指標。
- 監察 process runs。
- 審閱 audit logs。
- 管理 organization settings、users、billing、security 及 API keys。

frontend server 負責：

- Presentation。
- Interaction。
- Routing。
- Layout。
- UI state。
- Frontend validation。
- User experience。
- Client-side real-time updates。
- Design-system implementation。

frontend server 不得包含核心 backend 業務邏輯。

2. 產品願景

frontend 應呈現出一個嚴謹的企業級 SaaS 應用，用於營運由 AI 驅動的業務自動化。

它不應看起來像一個通用的 AI 示範頁。

介面應傳達：

- 信任。
- 可靠性。
- 營運清晰度。
- 安全性。
- 專業感。
- 速度。
- 控制力。
- 可觀測性。

使用者應始終清楚知道：

- 有哪些 agents。
- 有哪些 workflows。
- 甚麼正在執行。
- 甚麼執行失敗。
- 甚麼需要審批。
- 已連接哪些 knowledge sources。
- 使用了哪些資料。
- 採取了哪些動作。
- 誰做了甚麼。
- 甚麼需要關注。

3. 核心設計原則

frontend 必須透過有意識的設計流程來設計。

Trae 在實作主要頁面版面之前，必須先使用 **OpenDesign MCP**。

工作流程應為：

```
Trae
-> OpenDesign MCP
  -> Search design references
  -> Select a suitable design system
  -> Extract tokens and layout guidance
  -> Produce a layout plan
  -> Implement React/TypeScript/Tailwind frontend
```

frontend 不得只依靠通用 AI 記憶來設計。

4. 範圍

4.1 包含範圍

frontend server 必須包括：

- 公開入口頁或重新導向行為。
- Authentication 頁面。
- 已驗證的 app shell。
- Dashboard 頁面。
- Agents 區段。
- Tools 區段。
- Workflows 區段。
- Workflow run 詳情區段。
- Approvals 區段。
- Knowledge 區段。
- Memory 區段。
- Evaluations 區段。
- Processes 區段。
- Audit logs 區段。
- Organization settings。
- User settings。
- API key management UI。
- Security settings UI。
- Billing placeholder 或當 backend 支援時提供 billing page。
- 響應式版面。
- Loading states。
- Empty states。
- Error states。
- 具權限感知的導覽。
- Typed API integration。
- Real-time workflow run UI。
- 基於 OpenDesign 的設計文件。

4.2 不包含範圍

frontend 不得實作：

- Workflow execution logic。
- Agent execution logic。
- Tool execution logic。
- 作為唯一真相來源的 permission enforcement。
- 直接寫入 database。
- 背景 jobs。
- Embedding/indexing logic。
- Secret storage。
- 在 browser code 中處理 provider API key。
- 建立 audit log。
- Billing calculations。
- 作為最終權威的 authentication verification。

backend 仍然是以下事項的唯一真相來源：

- Authentication。
 - Authorization。
 - Permissions。
 - Workflow execution。
 - Agent execution。
 - Tool execution。
 - Knowledge indexing。
 - Memory storage。
 - Evaluation execution。
 - Governance rules。
 - Audit logging。
 - Organization security。
-

5. 建議技術棧

5.1 Framework

使用：

```
Next.js  
React  
TypeScript  
Tailwind CSS
```

建議架構：

```
Browser  
  -> Next.js Frontend Server  
    -> Backend API  
        -> FastAPI backend services
```

5.2 Package Manager

建議：

```
pnpm
```

可接受的替代方案：

```
npm  
yarn
```

5.3 Styling

使用：

```
Tailwind CSS  
Design tokens  
Reusable UI components
```

frontend 不應使用隨機硬編碼顏色。

顏色、字體排印、間距、圓角、陰影與動態規則，應來自透過 OpenDesign 產生或選定的 design system。

5.4 Forms

建議：

```
React Hook Form  
Zod
```

Form 規則：

- 在 client-side 進行驗證以提升可用性。
- backend 仍是最終驗證者。
- 清楚顯示 server validation errors。
- 在請求進行中停用 submit。
- 防止重複提交。

5.5 API Client

使用根據 backend OpenAPI schema 產生的 typed API client。

建議選項：

```
openapi-typescript  
orval  
openapi-fetch  
custom generated TypeScript client
```

frontend 不得手動猜測 backend request 與 response 的結構。

5.6 Real-Time Updates

使用：

```
SSE for workflow run updates  
WebSocket only if bidirectional live interaction is required
```

SSE 較適合用於：

- Workflow run timeline updates。
- Logs streaming。
- Approval waiting states。
- Step status updates。
- Process monitoring。

5.7 Testing

使用：

```
Playwright for end-to-end tests
Vitest or Jest for unit tests
React Testing Library for component tests
axe or equivalent accessibility checks
```

5.8 Deployment

frontend 應可在不使用 Docker 的情況下部署。

建議部署方式：

```
pnpm install
pnpm build
pnpm start
```

生產環境程序管理工具可包括：

```
systemd
PM2
managed Node hosting
Vercel-like platform
```

6. 執行期職責

6.1 Frontend Server 職責

frontend server 負責：

- Render 頁面。
- 導引使用者路由。
- 保護 frontend routes。

- 呼叫 backend APIs。
- 管理 UI session state。
- 顯示 backend 資料。
- 處理 frontend errors。
- 顯示 real-time updates。
- 呈現具角色感知的導航。
- 套用 design tokens。
- 提供 static assets。
- 提供可存取的 UI 行為。

6.2 Backend 職責

backend 負責：

- 使用者 authentication validation。
- Authorization。
- Roles 與 permissions。
- Agent persistence。
- Workflow persistence。
- Workflow execution。
- Tool execution。
- Approval state transitions。
- Audit log writes。
- Knowledge ingestion。
- Memory writes。
- Evaluation jobs。
- Organization governance。
- Secrets 與 credentials。

6.3 邊界規則

frontend 可以提出動作請求。

backend 決定該動作是否被允許。

例子：

```
Frontend:  
  User clicks "Run Workflow"
```

```
Backend:  
  Verifies permission  
  Creates run  
  Starts execution  
  Emits status events  
  Writes audit log
```

frontend 不得假設「隱藏按鈕」已足以構成安全控制。

7. 應用架構

7.1 高層架構

```
User Browser
|
| HTTPS
v
Next.js Frontend Server
|
| HTTPS / internal network
v
Backend API
|
+-- Auth service
+-- Agents service
+-- Tools service
+-- Workflows service
+-- Runs service
+-- Approvals service
+-- Knowledge service
+-- Memory service
+-- Evaluation service
+-- Audit service
+-- Organization service
```

7.2 Frontend Rendering 策略

使用混合式 rendering 方法：

- 適當情況下以 server-render app shell。
- 對高互動性 widgets 使用 client-render。
- 以下項目使用 client components：
 - Command palette。
 - Realtime run timeline。
 - Modals。
 - Drawers。
 - 帶篩選器的資料表。
 - Workflow builder。
 - Logs viewer。
- 以下項目使用 server components：
 - 靜態 layouts。
 - 可安全傳遞 cookies 的初始資料抓取。
 - Metadata。
 - 基本頁面組合。

7.3 路由保護

受保護 routes：

```
/app/*
```

規則：

- 如使用者未通過驗證，重新導向到 `/login`。
- 如使用者沒有 organization access，重新導向到 organization selection 或 access-denied page。
- 如使用者缺少某 route 所需權限，顯示 `403 Access Denied`。
- Route protection 只屬 UX 層級。
- Backend authorization 仍為最終裁決。

8. OpenDesign MCP 要求

8.1 強制要求

在 Trae 建立或大幅修改任何主要 frontend page layout 之前，Trae 必須呼叫名為以下的 MCP server：

```
opendesign
```

主要頁面版面包括：

- Dashboard。
- Agents list。
- Agent detail。
- Workflow list。
- Workflow builder。
- Workflow detail。
- Workflow run detail。
- Approvals queue。
- Knowledge dashboard。
- Evaluation dashboard。
- Audit logs。
- Settings。
- Billing。
- Security pages。

8.2 必需的 OpenDesign MCP 呼叫

對於每個主要頁面，Trae 必須執行以下 MCP 流程：

```
1. get_director_protocol
2. search_designs
3. get_design_system
4. fetch_design_spec_markdown
```

5. Summarize layout plan
6. Implement page

8.3 Trae 必需輸出的設計內容

在寫程式碼前，Trae 必須產出：

- Selected design reference
- Why the reference fits the page
- Extracted design tokens
- Typography approach
- Spacing approach
- Layout grid
- Component hierarchy
- Responsive behavior
- Accessibility concerns
- Implementation plan

8.4 後備規則

如果 OpenDesign MCP 無法使用：

1. Trae 必須停止。
2. Trae 必須回報 OpenDesign MCP 無法使用。
3. 除非專案擁有者明確批准後備方案，否則 Trae 不得繼續使用通用版面。
4. 如後備方案已獲批准，Trae 必須在以下位置記錄原因：

```
frontend/docs/design/open-design-reference.md
```

9. Trae 的 OpenDesign MCP 設定

9.1 MCP 檔案位置

把 MCP server 檔案存放於：

```
frontend/.mcp/opendesign-mcp.mjs
```

或於專案根目錄：

```
.mcp/opendesign-mcp.mjs
```

建議使用專案根目錄設定：

```
mkdir -p .mcp
```

從官方 OpenDesign 來源下載官方 OpenDesign MCP server 檔案，並儲存為：

```
.mcp/opendesign-mcp.mjs
```

9.2 Trae MCP 設定

建立：

```
.trae/mcp.json
```

使用絕對路徑。

macOS 或 Linux 範例：

```
{
  "mcpServers": {
    "opendesign": {
      "command": "node",
      "args": ["/ABSOLUTE/PATH/TO/YOUR/PROJECT/.mcp/opendesign-mcp.mjs"]
    }
  }
}
```

Windows 範例：

```
{
  "mcpServers": {
    "opendesign": {
      "command": "node",
      "args": ["C:\\ABSOLUTE\\PATH\\TO\\YOUR\\PROJECT\\.mcp\\opendesign-mcp.mjs"]
    }
  }
}
```

建立或編輯此檔案後：

Restart Trae.

9.3 MCP 安全規則

OpenDesign MCP server 應被視為具特權的開發工具。

規則：

- 只使用官方來源。
- 如可行，固定或記錄已下載檔案的版本。
- 不要向 MCP servers 提供不必要的 secrets。
- 不要向 MCP tools 暴露生產用 API keys。
- 提交前先審閱任何由 MCP 產生的版面或程式碼。
- 未經批准，不得允許 MCP tools 修改與工作無關的專案檔案。
- 記錄已選用的設計參考。
- 不要直接複製受專有權保護的資產。
- 參考版面靈感，而不是挪用資產。

10. Trae 專案規則

建立：

```
.trae/project_rules.md
```

使用以下內容：

Frontend Project Rules

Mandatory OpenDesign MCP Usage

Before implementing or redesigning any major frontend page layout, call the MCP server named `opendesign`.

Required MCP workflow:

1. Call `get\_director\_protocol`.
2. Call `search\_designs`.
3. Select a suitable SaaS, dashboard, developer-tool, workflow, or enterprise reference.
4. Call `get\_design\_system`.
5. Call `fetch\_design\_spec\_markdown`.
6. Extract design tokens, spacing, typography, layout structure, motion guidance, and component hierarchy.
7. Produce a short layout plan before writing code.
8. Implement the page using React, TypeScript, and Tailwind CSS.

Do not create generic dashboard UI from memory when OpenDesign MCP is available.

Frontend Architecture Rules

- Use TypeScript.

- Use Next.js App Router.
- Use Tailwind CSS.
- Use reusable components.
- Use semantic HTML.
- Use accessible components.
- Use typed backend API clients.
- Do not manually guess backend API shapes.
- Do not place backend business logic in frontend components.
- Do not expose secrets in browser code.
- Do not store sensitive auth tokens in localStorage.
- Prefer HttpOnly secure cookies for sessions.
- Add loading, empty, and error states for every major page.
- All pages must be responsive.

Design Rules

- Use OpenDesign-derived tokens.
- Do not hardcode random colors.
- Keep visual hierarchy clear.
- Prioritize enterprise SaaS clarity.
- Use status colors consistently.
- Make destructive actions obvious and confirmed.
- Preserve keyboard navigation.
- Preserve focus states.
- Ensure color contrast.
- Avoid generic AI-themed gradients unless justified by the selected design system.

Implementation Rules

- Keep files small and focused.
- Extract reusable UI primitives.
- Extract page-specific feature components.
- Add tests for critical flows.
- Run type checking before completion.
- Run linting before completion.
- Document assumptions.
- If backend endpoints are missing, create clear TODOs and API contract notes instead of inventing fake APIs.

11. 建議的 Trae Prompts

11.1 一般 Dashboard Prompt

Use the OpenDesign MCP server before writing code.

I am building the frontend for an AI business automation platform.

Design the main authenticated dashboard layout.

Requirements:

- Professional SaaS dashboard
- Left sidebar navigation
- Top command/search area
- Agent status cards
- Workflow run activity
- Approvals queue
- Knowledge base health
- Evaluation score summary
- Audit/security indicators
- Responsive desktop/tablet/mobile behavior
- Clean enterprise look
- Not generic AI dashboard style

First:

1. Call OpenDesign get_director_protocol.
2. Search OpenDesign for a suitable SaaS, AI, developer-tool, or workflow-dashboard reference.
3. Retrieve the design system and design spec.
4. Summarize the layout plan.
5. Then implement the page using React, TypeScript, and Tailwind CSS.

11.2 Workflow Detail Prompt

Use OpenDesign MCP to design the /app/workflows/[workflowId] page.

The page should include:

- Workflow title and metadata
- Workflow status
- Visual workflow steps
- Run button
- Latest run status
- Approval checkpoints
- Logs panel
- Error panel
- Version history
- Settings drawer

Call OpenDesign first, select a strong reference, summarize the layout plan, then implement the page.

11.3 Workflow Run Detail Prompt

Use OpenDesign MCP to design the /app/workflow-runs/[runId] page.

The page should include:

- Run status summary
- Timeline of steps
- Live logs

- Tool calls
- Agent messages
- Approval waiting state
- Error details
- Retry action
- Cancel action
- Final output panel
- Audit metadata

Use a professional operations-console style. Call OpenDesign first.

11.4 Knowledge Page Prompt

Use OpenDesign MCP to design the /app/knowledge page.

The page should include:

- Knowledge health summary
- Connected sources
- Recently indexed documents
- Failed ingestion jobs
- Search box
- Source type filters
- Index freshness indicators
- Add source action
- Document detail drawer

Use OpenDesign before implementation.

12. 資訊架構

12.1 主要路由

```
/
  Public landing page or redirect

/login
/register
/forgot-password
/reset-password
/accept-invite

/app
  Main dashboard

/app/agents
/app/agents/new
/app/agents/[agentId]
```

```
/app/tools
/app/tools/[toolId]

/app/workflows
/app/workflows/new
/app/workflows/[workflowId]

/app/workflow-runs/[runId]

/app/approvals
/app/approvals/[approvalId]

/app/knowledge
/app/knowledge/sources
/app/knowledge/sources/[sourceId]
/app/knowledge/documents
/app/knowledge/documents/[documentId]
/app/knowledge/search

/app/memory
/app/memory/[memoryId]

/app/evaluations
/app/evaluations/[evaluationId]
/app/evaluations/runs/[evaluationRunId]

/app/processes
/app/processes/[processId]

/app/audit-logs

/app/settings
/app/settings/organization
/app/settings/users
/app/settings/roles
/app/settings/billing
/app/settings/api-keys
/app/settings/security
/app/settings/integrations
/app/settings/profile
```

12.2 導覽群組

Sidebar 導覽應分組：

```
Main
  Dashboard
  Agents
  Workflows
  Approvals
```



```
Data
  Knowledge
  Memory

Quality
  Evaluations
  Processes

Security
  Audit Logs

Admin
  Settings
```

12.3 全域 Header

已驗證 app 的 header 應包括：

- Breadcrumbs。
- 全域搜尋或 command 按鈕。
- 如非 production，顯示 environment indicator。
- Organization switcher。
- Notifications。
- User menu。

12.4 Command Palette

快捷鍵：

```
Cmd/Ctrl + K
```

動作：

- Create agent。
- Create workflow。
- Search knowledge。
- Open recent workflow run。
- Review approvals。
- Invite user。
- Open API keys。
- Open audit logs。
- Open security settings。
- Start evaluation。
- Add knowledge source。

13. 使用者角色與權限

frontend 應支援具角色感知的 UI。

建議角色：

```
Owner
Admin
Developer
Operator
Reviewer
Viewer
Billing Manager
Security Auditor
```

13.1 權限範例

```
agents:read
agents:create
agents:update
agents:delete

tools:read
tools:create
tools:update
tools:delete

workflows:read
workflows:create
workflows:update
workflows:delete
workflows:run

runs:read
runs:cancel
runs:retry

approvals:read
approvals:approve
approvals:reject

knowledge:read
knowledge:create
knowledge:update
knowledge:delete
knowledge:index

memory:read
memory:delete

evaluations:read
evaluations:create
```

```
evaluations:run

audit_logs:read

settings:read
settings:update

users:read
users:invite
users:update
users:remove

billing:read
billing:update

api_keys:read
api_keys:create
api_keys:revoke

security:read
security:update
```

13.2 權限顯示規則

frontend 應：

- 隱藏使用者不能執行的動作。
- 在有幫助時顯示 **disabled** 狀態。
- 對 **disabled actions** 顯示清楚說明。
- 對受限制 routes 顯示 **access-denied** 頁面。
- 絕不依賴隱藏 UI 作為安全措施。

14. 核心版面設計

14.1 App Shell

已驗證 app 必須使用持久化 shell。

必需元素：

- Sidebar
- Header
- Main content region
- Breadcrumb
- Command palette trigger
- Organization switcher
- User menu
- Notification center
- Responsive mobile navigation

14.2 Desktop Layout

Desktop 應使用：

Left sidebar + top header + content region

建議行為：

- Sidebar 固定或 sticky。
- Main content 可捲動。
- Header sticky。
- Content width 按頁面自適應。
- 高密度資料頁可使用全寬。
- 詳情頁可使用雙欄布局。

14.3 Tablet Layout

Tablet 應使用：

- 可摺疊 sidebar。
- Header 持續可見。
- 需要時把 tables 轉為 card lists。
- 重要 actions 移到 overflow menus。

14.4 Mobile Layout

Mobile 應使用：

- Drawer navigation。
- 為底部安全區保留間距。
- 單欄內容。
- 以 cards 取代寬表格。
- 在適當情況下使用 sticky primary action。
- Logs 與 timelines 針對垂直捲動最佳化。

15. 視覺設計需求

15.1 視覺風格

frontend 應讓人感到：

- 現代。
- 企業級。
- 沉穩。
- 精準。
- 值得信賴。
- 營運導向。
- 資料豐富但不凌亂。

避免：

- 隨機漸變。
- 玩具化 AI 視覺。
- 過大的空白卡片。
- 過量動畫。
- 不一致的狀態顏色。
- 不清晰的層級。

15.2 Design Tokens

建立 token 檔案：

```
frontend/src/design/tokens.ts  
frontend/src/design/theme.ts  
frontend/docs/design/open-design-reference.md
```

Token 類別：

```
Color  
Typography  
Spacing  
Radii  
Shadows  
Borders  
Surfaces  
Status colors  
Motion  
Z-index  
Breakpoints
```

15.3 狀態顏色

使用一致的語義化狀態顏色：

```
Success  
Warning  
Error  
Info  
Neutral  
Running  
Pending  
Paused  
Cancelled  
Draft  
Published
```

狀態使用範例：

```
Workflow run succeeded -> Success
Workflow run failed -> Error
Workflow waiting approval -> Warning or Pending
Agent disabled -> Neutral
Tool unavailable -> Error
Knowledge source indexing -> Running
```

15.4 Typography

Typography 應支援：

- 頁面標題。
- 區段標題。
- 卡片標題。
- 指標標籤。
- 表格文字。
- 程式碼或 log 文字。
- Metadata。
- Empty state 文案。
- Error 文案。

以下內容使用 monospace typography：

- IDs。
- Logs。
- API keys。
- JSON snippets。
- Tool call payloads。
- Trace IDs。

15.5 Motion

Motion 應保持克制。

Motion 可用於：

- Drawer open/close。
- Command palette。
- Toasts。
- Skeleton loading。
- 狀態轉換。
- Timeline updates。

避免會分散營運監察注意力的 motion。

16. 頁面需求

16.1 公開 Root 頁面

Route :

```
/
```

目的 :

- 顯示公開 landing page，或把已驗證使用者重新導向至 `/app`。

需求 :

- 如已驗證，重新導向到 dashboard。
- 如未驗證，顯示簡單產品介紹或重新導向到 `/login`。
- 必須輕量。
- 不得暴露內部 app 資料。

16.2 Login 頁面

Route :

```
/login
```

目的 :

- 讓使用者登入。

必需 UI :

- Email 欄位。
- Password 欄位。
- Submit 按鈕。
- Forgot password 連結。
- 如允許註冊，顯示 Register 連結。
- Error message 區域。
- Loading state。
- 如適用，處理 organization invite。

狀態 :

```
Default  
Submitting  
Invalid credentials  
Account locked  
MFA required  
Server unavailable
```

安全規則：

- 不要把 tokens 儲存在 localStorage。
- 如 backend 支援，使用 secure cookie session flow。
- 除非 backend policy 允許，否則不要透露某 email 是否存在。

16.3 Register 頁面

Route：

```
/register
```

目的：

- 如有啟用，讓新使用者或新 organization 註冊。

必需 UI：

- Name。
- Email。
- Password。
- Organization name。
- 如需要，Terms checkbox。
- Submit 按鈕。
- Login 連結。

規則：

- 如公開註冊已停用，顯示只限邀請訊息。
- 為提升可用性，在 client-side 驗證 password。
- backend 執行最終驗證。

16.4 Dashboard 頁面

Route：

```
/app
```

目的：

- 提供營運總覽。

必需區段：

- Welcome/header summary
- Agent health cards
- Workflow run activity
- Pending approvals
- Knowledge base health
- Evaluation summary
- Recent audit events
- Process status
- Quick actions

建議的 dashboard cards：

Active Agents
Workflows Running
Pending Approvals
Failed Runs
Knowledge Sources
Evaluation Pass Rate
Security Alerts
Recent Tool Errors

主要 actions：

- Create agent。
- Create workflow。
- Add knowledge source。
- Review approvals。
- Run evaluation。

Empty state：

- 如未有 agents 或 workflows，顯示 onboarding checklist。

Dashboard 好主意：

Add an onboarding checklist:

1. Create first agent
2. Add a tool
3. Add knowledge source
4. Create workflow
5. Run workflow
6. Review results

16.5 Agents List 頁面

Route：

/app/agents

目的：

- 顯示 organization 內所有 agents。

必需 UI：

- 頁面標題。
- Create agent 按鈕。
- 搜尋。
- Filters。
- Agent cards 或 table。
- Status indicators。
- Last run 資訊。
- Tool 數量。
- Knowledge access indicator。
- Owner 或 creator。
- Updated date。

Filters：

Status
Capability
Tool access
Knowledge access
Owner
Created date

Agent 狀態：

Draft
Active
Disabled
Error
Archived

Empty state：

No agents yet.
Create your first agent to automate a business task.

16.6 Create Agent 頁面

Route :

```
/app/agents/new
```

目的 :

- 建立新的 AI agent。

必需 UI :

- Agent name。
- Description。
- Instructions。
- 如支援，提供 model/provider selection。
- Tool permissions。
- Knowledge access。
- Memory settings。
- Safety settings。
- Test prompt 區域。
- Save draft 按鈕。
- Create agent 按鈕。

Validation :

- Name 必填。
- Instructions 必填。
- Tool permissions 必須明確指定。
- 危險的 tool access 應顯示警告。

好主意 :

```
Add a guided setup wizard:  
1. Identity  
2. Instructions  
3. Tools  
4. Knowledge  
5. Safety  
6. Test
```

16.7 Agent Detail 頁面

Route :

```
/app/agents/[agentId]
```

目的：

- 檢視與管理單一 agent。

必需區段：

- Agent header
- Status
- Description
- Instructions
- Tools
- Knowledge access
- Memory settings
- Recent runs
- Evaluation results
- Audit activity
- Danger zone

Actions：

- Edit。
- Duplicate。
- Enable/disable。
- Run test。
- View runs。
- Delete/archive。

狀態：

- 正在載入 agent。
- 找不到 agent。
- Access denied。
- Agent 已封存。

16.8 Tools List 頁面

Route：

```
/app/tools
```

目的：

- 管理可供 agents 或 workflows 使用的 tools 與 integrations。

必需 UI：

- Tool catalog。
- 已連接 tools。
- Tool status。
- Required credentials indicator。
- Permission scope。
- Usage count。
- Last used。
- Error status。

Tool 類別：

```
Communication
CRM
Documents
Data
Internal API
Web
Files
Code
Human Approval
```

安全規則：

- 絕不顯示 secret values。
- 只顯示已遮罩的 credential metadata。

16.9 Tool Detail 頁面

Route：

```
/app/tools/[toolId]
```

目的：

- 查看及設定某個 tool。

必需區段：

- Tool overview。
- Configuration。
- Credentials status。
- Permission scopes。
- 使用此 tool 的 agents。
- 使用此 tool 的 workflows。
- Recent calls。

- Failure rate。
- Audit events。

Actions :

- Connect。
- Reconnect。
- Disable。
- Test tool。
- Rotate credentials。
- Remove。

16.10 Workflows List 頁面

Route :

/app/workflows

目的 :

- 查看與管理 workflows。

必需 UI :

- Workflow list。
- Create workflow 按鈕。
- 搜尋。
- Filters。
- Status。
- Trigger type。
- Last run status。
- Success rate。
- Owner。
- Updated date。

Workflow 狀態 :

Draft
Active
Paused
Error
Archived

Filters :

```
Status
Trigger type
Owner
Last run status
Created date
Updated date
```

Empty state :

```
No workflows yet.
Build your first workflow to automate a repeatable process.
```

16.11 Create Workflow 頁面

Route :

```
/app/workflows/new
```

目的 :

- 建立新的 workflow。

必需 UI :

- Workflow name。
- Description。
- Trigger selection。
- Step builder。
- Agent step。
- Tool step。
- Condition step。
- Approval step。
- Notification step。
- Test run。
- Save draft。
- Publish。

好主意 :

```
Support workflow templates:
- Customer support triage
- Invoice processing
- Lead enrichment
- Document review
```

- Compliance approval
- Daily report generation

16.12 Workflow Detail 頁面

Route :

```
/app/workflows/[workflowId]
```

目的 :

- 查看、編輯、執行及監察 workflow。

必需區段 :

- Header
- Status
- Metadata
- Visual step map
- Trigger configuration
- Step list
- Latest run summary
- Approval checkpoints
- Version history
- Settings
- Audit activity

Actions :

- Run now。
- Edit。
- Duplicate。
- Pause。
- Publish。
- Archive。
- View latest run。
- Create new version。

關鍵 UI 行為 :

- 執行 workflow 時應即時顯示回饋。
- 如 backend 已建立 run，重新導向或連結至 [/app/workflow-runs/\[runId\]](/app/workflow-runs/[runId])。

16.13 Workflow Run Detail 頁面

Route :

```
/app/workflow-runs/[runId]
```

目的 :

- 顯示即時 workflow run 執行狀況。

這是最重要的頁面之一。

必需區段 :

- Run header
- Status summary
- Timeline
- Current step
- Step details
- Live logs
- Tool calls
- Agent messages
- Input payload
- Output payload
- Approval state
- Error details
- Retry/cancel actions
- Audit metadata

Run 狀態 :

```
Queued
Running
Waiting for Approval
Succeeded
Failed
Cancelled
Timed Out
Partially Completed
```

Timeline 項目類型 :

```
Workflow started
Agent step started
Agent step completed
Tool call started
Tool call completed
Approval requested
```

```
Approval approved
Approval rejected
Condition evaluated
Error occurred
Retry started
Workflow completed
```

Real-time 行為：

- 連接 SSE endpoint。
- 顯示 reconnecting 狀態。
- 去除重複 events。
- 保留捲動位置。
- 允許使用者暫停 auto-scroll。
- 在 run 完成時顯示最終狀態。
- 處理連線中斷。

Actions：

- Cancel run。
- Retry failed step。
- Retry workflow。
- Approve pending step。
- Reject pending step。
- Download logs。
- Copy run ID。
- Copy trace ID。

好主意：

```
Add an operations-style split view:
Left: timeline
Right: selected step details and logs
```

16.14 Approvals List 頁面

Route：

```
/app/approvals
```

目的：

- 讓人工審閱待處理 approvals。

必需 UI：

- Pending approvals queue。
- 搜尋。
- Filters。
- Priority。
- Requesting workflow。
- Requesting agent。
- Created date。
- Due date。
- Risk level。
- Approve/reject actions。

Filters :

```
Status
Priority
Workflow
Agent
Risk level
Due date
Requester
```

Approval 狀態 :

```
Pending
Approved
Rejected
Expired
Cancelled
```

Empty state :

```
No approvals need review.
Everything is moving automatically.
```

16.15 Approval Detail 頁面

Route :

```
/app/approvals/[approvalId]
```

目的 :

- 審閱單一 approval request。

必需區段：

- Request summary。
- Requested action。
- Reason。
- Input data。
- Proposed output。
- Risk indicators。
- Related workflow run。
- 涉及的 agent 或 tool。
- Audit trail。
- Comment box。
- Approve button。
- Reject button。

安全要求：

- 高風險 approvals 應顯示更強的確認對話框。

16.16 Knowledge 頁面

Route：

```
/app/knowledge
```

目的：

- 顯示 knowledge base 總覽。

必需區段：

```
- Knowledge health summary
- Connected sources
- Indexed documents
- Recent ingestion jobs
- Failed indexing jobs
- Search knowledge
- Add source action
```

指標：

```
Total sources
Indexed documents
Failed documents
```

Last indexed time
Index freshness
Storage usage
Search success rate

16.17 Knowledge Sources 頁面

Route :

/app/knowledge/sources

目的 :

- 管理 knowledge sources。

必需 UI :

- Sources table。
- Source type。
- Status。
- Document count。
- Last sync。
- Error count。
- Add source button。

Source 狀態 :

Connected
Indexing
Failed
Paused
Disconnected

Source 類型 :

File upload
Google Drive
Notion
Confluence
SharePoint
Website
Database
API
S3
Internal documents

16.18 Knowledge Search 頁面

Route :

```
/app/knowledge/search
```

目的 :

- 搜尋已索引 knowledge。

必需 UI :

- Search input。
- Filters。
- Results list。
- Source badges。
- 如 backend 有提供，顯示 relevance score。
- Snippets。
- Document detail drawer。
- Copy citation/reference。

好主意 :

```
Add "Why this result?" metadata showing source, document, chunk, and last indexed date.
```

16.19 Memory 頁面

Route :

```
/app/memory
```

目的 :

- 檢視 agents 使用的 memory records。

必需 UI :

- Memory records table。
- 搜尋。
- Filters。
- Agent filter。
- 如適用，提供 user/session filter。

- Created date。
- Last used。
- 如允許，提供 delete action。

安全規則：

- 敏感 memory values 可能需要遮蓋處理。
- 由 backend 決定可見性。

16.20 Evaluations 頁面

Route：

```
/app/evaluations
```

目的：

- 查看及執行 evaluations。

必需 UI：

- Evaluation suites。
- Recent evaluation runs。
- Pass/fail summary。
- Score trend。
- 被測試的 agent 或 workflow。
- Run evaluation 按鈕。

指標：

```
Pass rate
Average score
Regression count
Failed test cases
Last run date
```

好主意：

```
Add "quality gate" indicators before publishing high-risk workflows.
```

16.21 Evaluation Detail 頁面

Route：

```
/app/evaluations/[evaluationId]
```

目的：

- 查看一個 evaluation suite。

必需區段：

- Test cases。
- Expected behavior。
- Latest results。
- Historical score。
- Failures。
- Run settings。
- Trigger evaluation。

16.22 Processes 頁面

Route：

```
/app/processes
```

目的：

- 監察 backend processes、jobs 及長時間任務。

必需 UI：

- Process list。
- Status。
- Type。
- Started time。
- Duration。
- Owner。
- Related workflow/run。
- Progress。
- Error message。

Process 狀態：

```
Queued  
Running  
Succeeded  
Failed  
Cancelled  
Retrying
```

16.23 Audit Logs 頁面

Route :

```
/app/audit-logs
```

目的 :

- 顯示 security 與 activity 歷史。

必需 UI :

- Audit table。
- 搜尋。
- Filters。
- Date range。
- Actor。
- Action。
- Resource。
- 如可用，顯示 IP/device metadata。
- 如允許，提供 export action。

Audit event 範例 :

```
User signed in
Agent created
Workflow published
Workflow run started
Approval approved
API key created
Tool credentials rotated
User invited
Permission changed
Knowledge source deleted
```

安全規則 :

- Audit logs 在 frontend 中必須為唯讀。
- 不得透過 frontend 變更 audit records。

16.24 Settings 頁面

Route :

```
/app/settings
```

目的：

- 主 settings hub。

區段：

```
Organization
Users
Roles
Billing
API Keys
Security
Integrations
Profile
```

16.25 Organization Settings

Route：

```
/app/settings/organization
```

必需 UI：

- Organization name。
- Logo。
- Slug。
- Timezone。
- Default language。
- 如支援，提供 data retention settings。
- Save 按鈕。

16.26 User Management

Route：

```
/app/settings/users
```

必需 UI：

- Users table。

- Invite user 按鈕。
- Role badges。
- Status。
- Last active。
- Remove user。
- Change role。

User 狀態：

Active
Invited
Suspended
Removed

16.27 API Keys 頁面

Route：

/app/settings/api-keys

必需 UI：

- API keys list。
- Create key。
- Key name。
- Scopes。
- Created date。
- Last used。
- Revoke action。

安全規則：

- 完整 API key 只在建立後顯示一次。
- 之後絕不再顯示完整 key。
- 使用遮罩顯示。
- Revoke 動作需確認。

16.28 Security Settings

Route：

/app/settings/security

必需 UI：

- MFA settings。
 - Session policy。
 - Password policy。
 - 如支援，提供 SSO settings。
 - Allowed domains。
 - 如支援，提供 audit export。
 - Security alerts。
-

17. 可重用元件

17.1 Layout Components

```
AppShell
Sidebar
SidebarItem
Header
Breadcrumbs
PageHeader
PageSection
ContentGrid
MobileNav
OrganizationSwitcher
UserMenu
NotificationCenter
CommandPalette
```

17.2 UI Components

```
Button
IconButton
Input
Textarea
Select
Checkbox
RadioGroup
Switch
Tabs
Card
Badge
StatusBadge
Tooltip
Popover
DropdownMenu
Dialog
Drawer
Toast
```

Skeleton
EmptyState
ErrorState
LoadingState
DataTable
Pagination
FilterBar
SearchInput
DateRangePicker
CodeBlock
LogViewer
Timeline
MetricCard
ProgressBar
Stepper
ConfirmDialog

17.3 Domain Components

AgentCard
AgentStatusBadge
AgentToolList
AgentRunList

WorkflowCard
WorkflowStatusBadge
WorkflowStepMap
WorkflowRunTimeline
WorkflowRunStatusHeader
WorkflowRunLogs
WorkflowVersionHistory

ApprovalCard
ApprovalRiskBadge
ApprovalDecisionPanel

KnowledgeSourceCard
KnowledgeHealthCard
KnowledgeSearchResult
DocumentDetailDrawer

EvaluationScoreCard
EvaluationRunTable
EvaluationFailureList

AuditLogTable
AuditEventBadge

ApiKeyTable
UserRoleBadge
PermissionMatrix

18. Frontend 資料夾結構

建議結構：

```
frontend/  
  app/  
    layout.tsx  
    page.tsx  
    globals.css  
  
    login/  
      page.tsx  
  
    register/  
      page.tsx  
  
    forgot-password/  
      page.tsx  
  
    reset-password/  
      page.tsx  
  
    accept-invite/  
      page.tsx  
  
  app/  
    layout.tsx  
    page.tsx  
  
    agents/  
      page.tsx  
      new/  
        page.tsx  
        [agentId]/  
          page.tsx  
  
    tools/  
      page.tsx  
      [toolId]/  
        page.tsx  
  
    workflows/  
      page.tsx  
      new/  
        page.tsx  
        [workflowId]/  
          page.tsx  
  
    workflow-runs/  
      [runId]/
```

```
    page.tsx

  approvals/
    page.tsx
    [approvalId]/
      page.tsx

  knowledge/
    page.tsx
    sources/
      page.tsx
      [sourceId]/
        page.tsx
    documents/
      page.tsx
      [documentId]/
        page.tsx
    search/
      page.tsx

  memory/
    page.tsx
    [memoryId]/
      page.tsx

  evaluations/
    page.tsx
    [evaluationId]/
      page.tsx
    runs/
      [evaluationRunId]/
        page.tsx

  processes/
    page.tsx
    [processId]/
      page.tsx

  audit-logs/
    page.tsx

  settings/
    page.tsx
    organization/
      page.tsx
    users/
      page.tsx
    roles/
      page.tsx
    billing/
      page.tsx
    api-keys/
      page.tsx
    security/
```

```
    page.tsx
  integrations/
    page.tsx
  profile/
    page.tsx

src/
  components/
    ui/
    layout/
    dashboard/
    agents/
    tools/
    workflows/
    approvals/
    knowledge/
    memory/
    evaluations/
    processes/
    audit/
    settings/

  design/
    tokens.ts
    theme.ts
    status.ts

  hooks/
    use-command-palette.ts
    use-permissions.ts
    use-realtime-run.ts
    use-toast.ts

  lib/
    api/
      client.ts
      generated/
      errors.ts
    auth/
      session.ts
      permissions.ts
    config/
      env.ts
    realtime/
      sse.ts
    routes/
      paths.ts
    formatting/
      dates.ts
      status.ts
    errors/
      app-error.ts

  stores/
```



```
command-palette-store.ts
notification-store.ts

types/
  domain.ts
  permissions.ts
  navigation.ts

docs/
  design/
    open-design-reference.md
    layout-decisions.md
    design-token-map.md

  api/
    frontend-api-contract.md

tests/
  e2e/
  unit/
  components/

.trae/
  mcp.json
  project_rules.md

.mcp/
  opendesign-mcp.mjs

package.json
tsconfig.json
tailwind.config.ts
next.config.js
postcss.config.js
playwright.config.ts
README.md
frontend.md
```

19. Environment Variables

19.1 Public Variables

只有以 `NEXT_PUBLIC_` 為前綴的變數可在 `browser code` 中使用。

允許公開的 `variables`：

```
NEXT_PUBLIC_APP_ENV
NEXT_PUBLIC_APP_NAME
NEXT_PUBLIC_API_BASE_URL
NEXT_PUBLIC_ENABLE_REGISTRATION
```

```
NEXT_PUBLIC_ENABLE BILLING
NEXT_PUBLIC_ENABLE SSO
```

19.2 Server-Only Variables

只限 server 使用的 variables 不得暴露到 browser。

```
BACKEND_API_INTERNAL_URL
SESSION_COOKIE_NAME
NODE_ENV
```

19.3 Frontend 禁止項

絕不可暴露：

```
DATABASE_URL
REDIS_URL
JWT_SECRET
OPENAI_API_KEY
ANTHROPIC_API_KEY
STRIPE_SECRET_KEY
WEBHOOK_SECRET
PROVIDER_CLIENT_SECRET
INTERNAL_SERVICE_TOKEN
ENCRYPTION_KEY
```

20. API 整合

20.1 Typed Client 要求

frontend 必須使用 typed API client。

產生的 types 應儲存於：

```
frontend/src/lib/api/generated/
```

API client wrapper：

```
frontend/src/lib/api/client.ts
```

20.2 API Error Handling

所有 API errors 都應映射為一致的 UI 行為。

Error 類型：

```
400 Validation error
401 Unauthorized
403 Forbidden
404 Not found
409 Conflict
422 Invalid input
429 Rate limited
500 Server error
503 Service unavailable
Network error
Timeout
```

UI 回應：

```
401 -> redirect to login
403 -> show access denied
404 -> show not found
409 -> show conflict message
422 -> show form validation errors
429 -> show rate-limit retry message
500 -> show error state
Network -> show retry option
```

20.3 API Request 規則

所有 API calls 必須：

- 使用 typed client。
- 在使用 cookie-based auth 時附帶 credentials。
- 處理 loading states。
- 處理 errors。
- 避免重複提交。
- 在適當情況下取消過時請求。
- 避免在 logs 中暴露 secrets。

21. Real-Time Workflow Run Updates

21.1 SSE Client

建立：

```
frontend/src/lib/realtime/sse.ts  
frontend/src/hooks/use-realtime-run.ts
```

此 hook 應處理：

- Connection open。
- Message events。
- Error events。
- Reconnect attempts。
- Backoff。
- Final state close。
- Duplicate event IDs。
- Manual disconnect。

21.2 Run Event Shape

預期的 frontend event model：

```
type WorkflowRunEvent = {  
  id: string;  
  runId: string;  
  type:  
    | "run.started"  
    | "run.status_changed"  
    | "step.started"  
    | "step.completed"  
    | "step.failed"  
    | "tool_call.started"  
    | "tool_call.completed"  
    | "approval.requested"  
    | "approval.approved"  
    | "approval.rejected"  
    | "log"  
    | "error"  
    | "run.completed"  
    | "run.failed"  
    | "run.cancelled";  
  timestamp: string;  
  payload: Record<string, unknown>;  
};
```

21.3 Realtime UI 規則

UI 必須：

- 顯示即時狀態。
- 顯示連線狀態。
- 顯示 reconnecting 狀態。

- 不遺失已接收 events。
- 避免重複 events。
- 允許手動重新整理。
- 當 run 達到最終狀態時停止重新連線。

最終狀態：

```
Succeeded
Failed
Cancelled
Timed Out
```

22. Loading、Empty 與 Error States

每個主要頁面都必須包括：

```
Loading state
Empty state
Error state
Permission denied state
Not found state where applicable
```

22.1 Loading State

以下項目使用 skeletons：

- Dashboard cards。
- Tables。
- Detail headers。
- Timeline rows。
- Logs panels。

22.2 Empty State

Empty states 必須包括：

- 清晰標題。
- 簡短說明。
- 主要 action。
- 可選的次要 action。
- 如與 design system 一致，提供有用插圖或 icon。

例子：

No workflows yet
Create your first workflow to automate a repeatable business process.
[Create workflow]

22.3 Error State

Error states 必須包括：

- 友善說明。
- 如可行，提供 retry button。
- 如可用，提供技術參考 ID。
- 如適用，提供支援或聯絡選項。

23. 無障礙需求

frontend 必須以 WCAG AA 級無障礙為目標。

必需項目：

- Semantic HTML。
- Keyboard navigation。
- 可見的 focus states。
- 足夠的色彩對比。
- 在需要時使用 ARIA labels。
- 無障礙 dialogs。
- 無障礙 dropdowns。
- 無障礙 command palette。
- 正確的 form labels。
- 與欄位關聯的 error messages。
- 不可只靠顏色傳達狀態。
- 支援 reduced-motion。
- 對 screen reader 友善的 loading states。

具體要求：

- Sidebar navigation 必須可用鍵盤操作。
- Command palette 開啟時必須 trap focus。
- Modals 與 drawers 關閉時必須恢復 focus。
- Data tables 必須有具意義的 headers。
- Status badges 必須包含文字，而不只顏色。
- Timeline items 必須可被 screen readers 閱讀。

24. Security Requirements

24.1 Authentication

建議：

```
HttpOnly Secure SameSite cookies
```

避免：

```
localStorage token storage  
sessionStorage token storage  
browser-exposed long-lived secrets
```

24.2 Authorization

frontend 必須：

- 根據 permissions 顯示 UI。
- 仍然呼叫 backend 進行最終授權判定。
- 妥善處理 403 Forbidden。

24.3 XSS Protection

規則：

- 不要渲染來自 backend 的原始 HTML，除非已消毒。
- 對使用者產生內容做 escaping。
- 如需要，使用安全的 Markdown rendering。
- 不要注入不受信任 scripts。
- 避免使用 dangerouslySetInnerHTML。

24.4 CSRF

如使用 cookie-based auth：

- backend 應提供 CSRF 保護。
- 如需要，frontend 應傳送 CSRF token header。

24.5 Secret Handling

frontend 絕不可顯示或記錄 secrets。

敏感欄位：

```
API keys  
OAuth tokens  
Provider credentials  
Internal service tokens  
Webhook secrets  
Private documents  
Memory records marked sensitive
```

24.6 破壞性動作確認

以下動作必須要求確認：

- Delete agent。
- Archive workflow。
- Delete knowledge source。
- Delete memory。
- Revoke API key。
- Remove user。
- Disable security feature。
- Cancel running workflow。

高風險動作應要求輸入文字確認。

25. 效能需求

25.1 頁面效能

frontend 應優化：

- Initial load。
- Route transitions。
- Dashboard rendering。
- 大型 tables。
- Logs rendering。
- Timeline rendering。
- Search responsiveness。

25.2 Techniques

使用：

- Code splitting。
- Lazy loading。
- Pagination。
- 如有需要，對 logs 使用 virtualized lists。
- Debounced search。
- Request cancellation。
- Server-side filtering。
- 只在安全情況下使用 optimistic updates。
- 對高成本 UI 計算使用 memoization。

25.3 大型資料規則

對於大型資料：

- 不要一次載入全部 audit logs。
- 不要一次載入全部 workflow runs。

- 如 backend 支援 pagination，不要一次載入全部 logs。
 - 在可行情況下使用 cursor pagination。
 - 使用 server-side search 與 filters。
-

26. 可觀測性

frontend 應包含 observability hooks。

追蹤：

- 頁面載入失敗。
- API failures。
- Realtime connection failures。
- Unhandled exceptions。
- 緩慢頁面。
- 失敗的表單提交。
- Command palette 使用情況。
- Workflow run page errors。

不要追蹤：

- Secrets。
- 如屬敏感，不要追蹤完整 prompts。
- Private documents。
- API keys。
- 原始 memory content，除非已明確允許且完成消毒。

建議 events：

```
frontend.page.viewed
frontend.api.error
frontend.form.submitted
frontend.workflow_run.opened
frontend.workflow_run.realtime_disconnected
frontend.approval.approved
frontend.approval.rejected
frontend.command_palette.opened
```

27. 測試需求

27.1 Unit Tests

測試：

- Utility functions。
- Permission helpers。
- API error mapping。

- Status formatting。
- Date formatting。
- Realtime event reducers。

27.2 Component Tests

測試：

- Buttons。
- Forms。
- Dialogs。
- Data tables。
- Status badges。
- Empty states。
- Error states。
- Workflow timeline。
- Approval panel。

27.3 End-to-End Tests

關鍵 E2E 流程：

```
Login
Dashboard loads
Create agent
Create workflow
Run workflow
View workflow run updates
Approve pending approval
Search knowledge
Invite user
Create API key
Revoke API key
View audit logs
Logout
```

27.4 Accessibility Tests

對以下頁面執行 accessibility checks：

- Login。
- Dashboard。
- Workflow run detail。
- Approvals。
- Settings。
- Dialogs。
- Command palette。

28. 實作計劃

Phase 0: 探索與合約

Goals :

- 確認 backend API routes。
- 確認 auth model。
- 確認 permissions model。
- 確認 OpenAPI schema。
- 確認 SSE 或 WebSocket events。
- 確認 entity names 與 IDs。
- 確認部署目標。

Deliverables :

```
docs/api/frontend-api-contract.md
docs/design/open-design-reference.md
Initial route map
Initial navigation map
```

Exit criteria :

- 已提供 Backend OpenAPI schema。
- Auth flow 已文件化。
- 主要 entities 已文件化。
- Realtime event shape 已文件化。

Phase 1: 專案設定

Tasks :

1. Create Next.js TypeScript app.
2. Install Tailwind CSS.
3. Configure ESLint.
4. Configure Prettier.
5. Configure path aliases.
6. Configure environment loader.
7. Create base folder structure.
8. Add base app layout.
9. Add global CSS.
10. Add design token files.
11. Add README startup instructions.

Commands :

```
pnpm install
pnpm dev
pnpm build
pnpm lint
pnpm typecheck
```

Exit criteria :

- App 可在本機啟動。
- TypeScript 正常運作。
- Tailwind 正常運作。
- Lint 正常運作。
- Build 成功。

Phase 2: Trae 與 OpenDesign 設定

Tasks :

1. Add `.mcp/opendesign-mcp.mjs`.
2. Add `.trae/mcp.json`.
3. Add `.trae/project_rules.md`.
4. Restart Trae.
5. Test MCP server availability.
6. Ask Trae to call `get_director_protocol`.
7. Document selected design process.

Deliverables :

```
.trae/mcp.json
.trae/project_rules.md
docs/design/open-design-reference.md
docs/design/design-token-map.md
```

Exit criteria :

- Trae 可呼叫 OpenDesign MCP。
- Trae 會在主要 layouts 之前使用 OpenDesign。
- 設計參考已文件化。

Phase 3: Design System Foundation

Tasks :

1. Extract design tokens from OpenDesign.
2. Create color tokens.
3. Create typography tokens.
4. Create spacing tokens.
5. Create status tokens.
6. Create base UI components.
7. Create layout components.
8. Create loading/empty/error components.
9. Create status badge system.
10. Create form components.

優先實作的 base components：

Button
Input
Textarea
Select
Card
Badge
StatusBadge
Dialog
Drawer
Dropdown
Skeleton
EmptyState
ErrorState
DataTable
Toast
Tooltip
Tabs

Exit criteria：

- 基本 UI system 完成。
- Components 使用 tokens。
- Components 符合無障礙要求。
- Components 具回應式設計。

Phase 4: Authentication 與 Session UI

Tasks：

1. Implement login page.
2. Implement register page if enabled.
3. Implement forgot password page.
4. Implement reset password page.
5. Implement invite acceptance page.

6. Implement session fetch.
7. Implement protected route handling.
8. Implement logout.
9. Implement user menu.
10. Implement unauthorized and forbidden states.

Exit criteria :

- 使用者可登入。
- 使用者可登出。
- Protected routes 會正確重新導向。
- App shell 顯示目前使用者。
- 已處理 Unauthorized API errors。

Phase 5: App Shell 與導航

Tasks :

1. Build AppShell.
2. Build Sidebar.
3. Build Header.
4. Build Breadcrumbs.
5. Build OrganizationSwitcher.
6. Build UserMenu.
7. Build NotificationCenter placeholder.
8. Build CommandPalette.
9. Add responsive mobile navigation.
10. Add permission-aware nav filtering.

Exit criteria :

- `/app` routes 共用相同 shell。
- Sidebar 在 desktop 可正常運作。
- Drawer navigation 在 mobile 可正常運作。
- Command palette 可透過 Cmd/Ctrl + K 開啟。
- Navigation 會遵守 permissions。

Phase 6: Dashboard

Tasks :

1. Use OpenDesign MCP for dashboard layout.
2. Build dashboard data fetch.
3. Build metric cards.
4. Build workflow activity widget.
5. Build approvals widget.

6. Build knowledge health widget.
7. Build evaluation summary widget.
8. Build audit/security widget.
9. Build onboarding checklist.
10. Add loading, empty, and error states.

Exit criteria :

- Dashboard 可清楚提供營運總覽。
- 空白 organization 會看到 onboarding。
- Data cards 具回應式設計。
- 設計已文件化。

Phase 7: Agents 與 Tools

Tasks :

1. Build agents list.
2. Build create agent flow.
3. Build agent detail page.
4. Build agent edit form.
5. Build tool list.
6. Build tool detail page.
7. Build tool connection states.
8. Add permission-aware actions.
9. Add loading/empty/error states.
10. Add tests for basic flows.

Exit criteria :

- 使用者可查看 agents。
- 如有權限，使用者可建立 agent。
- 使用者可檢視 agent details。
- 使用者可查看 tools。
- Secrets 絕不暴露。

Phase 8: Workflows

Tasks :

1. Use OpenDesign MCP for workflow pages.
2. Build workflows list.
3. Build create workflow page.
4. Build workflow detail page.
5. Build visual step map.
6. Build workflow settings drawer.

7. Build version history UI.
8. Build run workflow action.
9. Redirect to run detail after run starts.
10. Add loading/empty/error states.

Exit criteria :

- 使用者可查看 workflows。
- 如有權限，使用者可建立 workflow。
- 使用者可檢視 workflow details。
- 如有權限，使用者可啟動 workflow run。

Phase 9: Workflow Run Realtime UI

Tasks :

1. Build SSE client.
2. Build useRealtimeRun hook.
3. Build run status header.
4. Build timeline.
5. Build logs viewer.
6. Build tool call viewer.
7. Build agent message viewer.
8. Build approval waiting panel.
9. Build error details panel.
10. Build cancel/retry actions.
11. Add reconnect handling.
12. Add final state behavior.

Exit criteria :

- Run page 會即時更新。
- Reconnect 行為正常。
- Logs 易於閱讀。
- Timeline 清晰。
- 待審批項目清楚可見。
- Failed runs 會顯示有用的 error details。

Phase 10: Approvals

Tasks :

1. Build approvals queue.
2. Build approval filters.
3. Build approval detail page.
4. Build approve action.

5. Build reject action.
6. Build risk badges.
7. Build confirmation dialogs.
8. Link approvals to workflow runs.
9. Add audit metadata display.
10. Add loading/empty/error states.

Exit criteria :

- Reviewer 可進行 approve/reject。
- 高風險 approvals 有清晰標示。
- Approval decisions 會顯示確認流程。
- Empty state 有實際用途。

Phase 11: Knowledge 與 Memory

Tasks :

1. Build knowledge overview.
2. Build knowledge sources list.
3. Build add source UI.
4. Build source detail page.
5. Build documents list.
6. Build document detail page.
7. Build knowledge search page.
8. Build memory list.
9. Build memory detail page.
10. Add redaction support.

Exit criteria :

- 使用者可檢視 knowledge health。
- 使用者可搜尋 knowledge。
- 使用者可查看 source status。
- Sensitive memory 已安全處理。

Phase 12: Evaluations 與 Processes

Tasks :

1. Build evaluations list.
2. Build evaluation detail.
3. Build evaluation run detail.
4. Build score cards.
5. Build failure list.
6. Build processes list.

7. Build process detail page.
8. Add filters.
9. Add loading/empty/error states.
10. Add tests.

Exit criteria :

- 使用者可查看品質指標。
- 使用者可檢視 failed evaluations。
- 使用者可監察 processes。

Phase 13: Audit Logs 與 Settings

Tasks :

1. Build audit logs table.
2. Add audit filters.
3. Build settings hub.
4. Build organization settings.
5. Build users page.
6. Build roles page.
7. Build API keys page.
8. Build security settings.
9. Build billing page or placeholder.
10. Build integrations page.

Exit criteria :

- Admin 可管理 organization settings。
- API key UI 安全。
- Audit logs 可讀且可篩選。
- Settings 遵守 permissions。

Phase 14: 品質、安全與發佈

Tasks :

1. Run full typecheck.
2. Run lint.
3. Run unit tests.
4. Run component tests.
5. Run E2E tests.
6. Run accessibility checks.
7. Review security.
8. Review performance.
9. Review responsive layouts.
10. Build production bundle.

11. Deploy staging.
12. Test staging.
13. Deploy production.

Exit criteria :

- Build 通過。
- Tests 通過。
- 關鍵流程通過。
- 沒有 browser-exposed secrets。
- Mobile layout 正常。
- Production deployment 正常。

29. 可加入的良好產品想法

29.1 Onboarding Checklist

針對新 organizations :

```
Create first agent
Connect first tool
Add knowledge source
Create first workflow
Run first workflow
Invite teammate
Review audit logs
```

29.2 Command Palette

進階使用者導航 :

```
Cmd/Ctrl + K
```

應支援 :

- Navigation。
- Creation actions。
- Recent items。
- Search。
- Admin shortcuts。

29.3 Recent Activity Feed

Dashboard 應顯示 :

- Recent workflow runs。

- Failed tool calls。
- New approvals。
- User invitations。
- Security changes。
- Knowledge syncs。

29.4 Risk Indicators

在以下項目顯示風險：

- Approval requests。
- Tool calls。
- Workflow publishing。
- API key scopes。
- Security settings。

風險等級：

Low
Medium
High
Critical

29.5 Templates

提供以下 templates：

- Agents。
- Workflows。
- Evaluations。
- Knowledge sources。

29.6 Saved Views

對於 tables：

- 儲存 filters。
- 儲存 search。
- 儲存 column visibility。
- 如 backend 支援，與團隊分享檢視。

29.7 Explainability Drawer

對於 workflow runs：

Why did this happen?
What data was used?
Which agent made the decision?

Which tools were called?
Which approval rule applied?

29.8 Quality Gates

在發佈 workflows 前：

- 顯示最新 evaluation score。
- 顯示 failed tests。
- 如未有 evaluation，顯示警告。
- 如 workflow 使用高風險 tools，顯示警告。

30. 驗收標準

frontend server 在以下情況下視為完成：

- Frontend starts without Docker.
- Next.js app builds successfully.
- Trae can connect to OpenDesign MCP.
- Trae calls OpenDesign before major page layouts.
- Design references are documented.
- App shell exists.
- Login flow exists.
- Protected routes work.
- Dashboard exists.
- Agents pages exist.
- Tools pages exist.
- Workflows pages exist.
- Workflow run detail page supports live status UI.
- Approvals pages exist.
- Knowledge pages exist.
- Memory pages exist.
- Evaluations pages exist.
- Processes pages exist.
- Audit logs page exists.
- Settings pages exist.
- Command palette exists.
- All major pages are responsive.
- All major pages have loading states.
- All major pages have empty states.
- All major pages have error states.
- API calls use typed client.
- Permissions affect UI.
- Backend remains final source of authorization.
- No backend secrets are exposed.
- API keys are masked.
- Destructive actions require confirmation.
- Accessibility checks pass for critical pages.
- E2E tests pass for critical flows.

31. 每個頁面的完成定義

一個頁面只有在以下情況才算完成：

- OpenDesign MCP was used if the page is a major layout.
- Layout plan was documented.
- Page is implemented with TypeScript.
- Page uses shared layout components.
- Page uses design tokens.
- Page is responsive.
- Page has loading state.
- Page has empty state.
- Page has error state.
- Page handles permission restrictions.
- Page uses typed API client.
- Forms validate input.
- Server errors display clearly.
- Important actions have confirmation.
- Accessibility basics are satisfied.
- Page passes lint and typecheck.
- Critical interactions are tested.

32. 最終 Frontend 規則

frontend server 應是一個乾淨、專業的 SaaS 介面，用於營運 AI 業務自動化。

設計流程屬強制要求：

```
Trae
-> OpenDesign MCP
  -> Choose real design reference
  -> Extract design tokens
  -> Create layout plan
  -> Implement frontend page
```

frontend 必須：

```
Secure
Typed
Responsive
Accessible
Observable
Permission-aware
Operationally clear
Design-system driven
```

frontend 不得：

- Generic
- Toy-like
- Secret-leaking
- Backend-logic-heavy
- Unstructured
- Hardcoded
- Inaccessible
- Unclear during failures